OpenSpeX Whitepaper v2.0

Opportunity Infrastructure for Proven Performance

Prove ability. Build trust. Earn opportunity.

Prepared by D.A.R.M.A.
Research-grounded draft
17 June 2026

This document distinguishes current foundations, MVP targets, planned systems, experimental concepts, and long-term vision.

Important notice

This document is a strategic, technical, and product whitepaper for OpenSpeX, the emerging coordination and reputation layer built on the SpaceXpanse and ROD foundation. It is intended for ecosystem design, product planning, developer alignment, institutional discussion, and community education.

This document is not legal, tax, accounting, financial, investment, or securities advice. ROD, names, digital identities, NFTs, reward records, reputation signals, and other digital assets or records may carry technical, regulatory, custody, market, operational, and governance risks. Nothing in this document should be read as a promise of token value, investment return, passive income, regulatory approval, complete decentralization, or guaranteed implementation.

Several OpenSpeX and SpaceXpanse components are already grounded in existing artifacts, codebases, documentation, or prototypes. Others are design concepts, planned modules, research directions, or roadmap items. This whitepaper deliberately separates current foundations, near-term MVP work, and longer-term experimental layers. Where a feature is not already production-ready, it is marked as **MVP**, **planned**, **experimental**, or **research**.

Table of contents

---

1. Executive summary

OpenSpeX is an opportunity network powered by proven performance. It helps people, AI agents, validators, creators, service providers, businesses, researchers, and digital communities earn trust, access, rewards, and future opportunities by proving what they can actually do.

The internet has made it easy to claim ability. People claim skills. Platforms claim fairness. Companies claim reliability. AI systems claim intelligence. Service providers claim performance. But claims are cheap. They can be exaggerated, copied, bought, automated, gamed, or made to look real without being real. OpenSpeX is designed to close the gap between what is claimed and what can be proven. Its foundation is the principle stated in the OpenSpeX Manifesto: proof becomes reputation, and reputation becomes opportunity. [I-MANIFESTO]

The simplest OpenSpeX loop is:

```
Identity
-> Task
-> Evidence
-> Verification
-> Reputation
-> Settlement
-> Opportunity
```

A person completes a task. An AI agent solves a challenge. A service delivers an output. A creator publishes an asset. A validator reviews a result. A student completes a simulation mission. A digital system records what happened, verifies the outcome against clear rules, updates a performance record, and optionally settles reward or payment. The result should not disappear inside one app, one game, one employer, one platform, one leaderboard, or one benchmark. It should become reusable proof.

OpenSpeX is not a replacement for SpaceXpanse. It is the next abstraction layer above it. SpaceXpanse remains the historical, technical, blockchain, simulator, developer, and community foundation. ROD remains the settlement, ownership, naming, identity, asset, and reputation anchor. OpenSpeX makes those foundations legible as infrastructure for verified digital work, AI-agent reputation, creator economies, simulation credentials, and ownable digital ecosystems. [I-EVOLUTION] [I-LAUNCH]

In the old framing, SpaceXpanse was presented as a next-generation trustless GameFi blockchain metaverse platform. That was not wrong, but it was too narrow. The stronger current framing is infrastructure first: SpaceXpanse is the open foundation for ownable digital economies, while OpenSpeX is the coordination and reputation layer that turns actions into evidence, evidence into reputation, and reputation into opportunity. [I-EVOLUTION]

The first proof environments are intentionally concrete. **Void Runner Arena** is the lightweight browser-native proof environment: fast, public, measurable, accessible, and suitable for human-vs-AI performance comparison. **Orbit Challenge** is the deeper simulation-native proof environment: telemetry-rich, physics-based, educational, and institutionally credible. These are not merely games. They are digital flight tests. They demonstrate the larger principle: do not tell us what you can do; fly the mission, complete the task, produce the record, and let the evidence speak. [I-VOID] [I-FLIGHT]

The long-term market category is **Verified Digital Economy Infrastructure**: an open coordination layer that connects people, AI agents, services, tasks, evidence, reputation, settlement, and opportunity across interoperable digital ecosystems. OpenSpeX does not compete as another metaverse, another crypto token, another AI wrapper, another cloud marketplace, or another closed freelance platform. It competes as infrastructure for a world where humans and AI systems increasingly perform digital work, but trust must be earned from evidence rather than claims. [I-LAUNCH]

The immediate execution doctrine is strict: prove one loop, then another, then connect them. The first undeniable loop should be ROD-backed identity, a measurable challenge, evidence capture, validation, leaderboard or reputation update, ROD/testnet settlement or badge, and a public profile shown through Launchpad or a lightweight web dashboard. Large 3D worlds, full DAO governance, high-stakes slashing, large-scale decentralized compute, regulated credentials, enterprise identity, and autonomous AI-agent markets should remain roadmap or research items until the early proof loops are stable. [I-LAUNCH] [I-MVP]

OpenSpeX can become valuable because verified performance compounds. A participant with a long history of validated completions, clear failure modes, reliable latency, dispute history, cost-performance data, and domain-specific proof becomes more valuable than a profile full of claims. The real long-term asset is not one game, one token, one benchmark, or one marketplace. It is the accumulated verified reputation graph across humans, AI agents, creators, services, simulations, tasks, and worlds.

---

2. The core problem

2.1 The digital economy runs on claims

Most online opportunity is still allocated through claims. A resume claims experience. A profile claims expertise. A platform rating claims reliability. A certificate claims competence. A benchmark claims intelligence. A review claims satisfaction. A screenshot claims completion. A leaderboard claims performance. A vendor demo claims capability.

Some claims are honest. Many are incomplete. Some are misleading. Some are fabricated. Even when claims are true, they are often locked inside platforms that own the data, control the ranking rules, and decide who gets access to opportunity.

This creates a structural trust problem:

- The person who performed useful work may not own the record.
- The business that needs reliable help may not know who can actually deliver.
- The AI builder may not be able to prove that an agent works outside a demo.
- The validator may not be rewarded for careful judgment.
- The creator may not convert useful contributions into durable reputation.
- The user may have to start from zero whenever they move to a new platform.
- The platform may become the gatekeeper of trust instead of merely a place where work happens.

OpenSpeX is built around a different premise: opportunity should come from what can be proven, not merely from what can be claimed.

2.2 Platform-owned reputation creates platform reset

Closed platforms make reputation useful inside their own walls, but weak outside them. A freelancer can accumulate years of ratings on one marketplace and still start from zero elsewhere. A microtask worker can complete thousands of tasks without owning a portable record of reliability. A gamer can show skill in one leaderboard without that skill becoming useful in broader digital work. An AI agent can perform well in one private test without earning operational trust in the wider ecosystem. [I-OVERVIEW]

This is the platform reset problem. Every closed reputation system creates a local history, but the performer often cannot carry that history as a durable, independently verifiable signal.

OpenSpeX does not need to replace every platform to improve this. It can begin by creating portable performance records for controlled environments where evidence can be captured and verified. Over time, those records can become useful across tasks, games, simulations, AI-agent evaluations, microtask markets, creator economies, and service networks.

2.3 AI agents intensify the trust problem

AI agents are beginning to browse, write code, operate tools, extract data, review documents, generate media, coordinate workflows, and make delegated decisions. This changes the trust problem. The question is no longer only, "Who are you?" It becomes:

- What have you actually completed?
- Under what rules?
- With what evidence?
- How often do you fail?
- What failure modes are common?
- What is the cost of a correct result?
- How does the agent perform under hidden edge cases?
- Can the result be reproduced or audited?
- Who validated the output?
- Can the agent be trusted with a larger task?

The market is already moving from static question-answering toward agentic execution. Recent and emerging benchmarks increasingly test whether agents can complete realistic workflows, not merely produce plausible answers. OpenAI has publicly explained why it no longer reports SWE-bench Verified as a core progress metric, citing issues such as flawed tests and benchmark contamination; this supports the need for fresher, more dynamic, more operational evaluations. [E-OPENAI-SWE] Newer benchmark work such as OccuBench and JobBench focuses on real-world professional tasks, long-horizon workflows, robustness, and occupational delegation rather than static trivia-style evaluation. [E-OCCUBENCH] [E-JOBBENCH]

OpenSpeX is not merely an AI benchmark. It is broader: it can turn completed tasks into operational reputation. But AI-agent evaluation is one of its strongest initial wedges because the trust gap is immediate, visible, and commercially important.

2.4 Static benchmarks decay

A benchmark can become less useful over time. It may be overfit. It may be contaminated. It may become known to model developers. It may test the wrong behavior. It may measure completion in unrealistic conditions. It may reward answer generation instead of robust execution.

OpenSpeX addresses this through dynamic proof environments rather than static datasets alone. A task can be generated, seeded, varied, validated, and archived. A performance record can include context, timing, cost, attempts, mistakes, corrections, validator feedback, and success criteria. SpeXcode can later strengthen this by generating new tasks, scenarios, workflows, missions, and hidden edge cases. [I-SPEXCODE]

2.5 Digital work lacks a shared flight recorder

The flight recorder analogy is central. Aviation learned that trust cannot rely only on confidence, status, or verbal reports. Important actions are measured. Telemetry exists. Records can be reviewed. Failures can be understood. Training can improve through evidence. [I-FLIGHT]

OpenSpeX applies the same principle to digital performance. It acts like an open flight recorder for digital work: not a closed black box that is only opened after failure, but a performance record that can help participants earn trust while opportunity is still alive.

For OpenSpeX:

```
Task
-> Evidence
-> Open performance record
-> Verification
-> Reputation
-> Opportunity
```

Void Runner Arena and Orbit Challenge are early proof environments precisely because flight-like tasks make the principle intuitive. A mission either happened or it did not. A trajectory can be inspected. A score can be validated. A run can produce telemetry. A result can become part of a reputation profile.

2.6 Infrastructure fragmentation blocks ownable digital economies

The broader SpaceXpanse thesis is that digital economies need shared rails for identity, ownership, payments, communication, discovery, creator tooling, simulation, reputation, and settlement. Without shared rails, every game, marketplace, AI service, or virtual world must rebuild its own trust system. This creates fragmentation, lock-in, duplicated development, and weak interoperability.

SpaceXpanse already contains or proposes many of these rails: ROD blockchain, SpeXID, ROD names and decentralized discovery, tokens/NFTs, atomic trading, RODPay/Democrit, Nostr/Metalog, SpeX Library, DappEngine, Launchpad, Metaverse Simulator, SpeXcode, and D.A.R.M.A. [I-EVOLUTION] OpenSpeX converts these rails into a clearer public category: verified performance, operational reputation, and opportunity infrastructure.

---

3. The OpenSpeX thesis

3.1 One-sentence thesis

OpenSpeX helps people and AI agents earn opportunity through proven performance.

A more technical version:

OpenSpeX is open coordination and reputation infrastructure for verified digital work, AI-agent performance, creator economies, simulation credentials, and ownable digital ecosystems, built on the SpaceXpanse and ROD foundation.

An institutional version:

OpenSpeX is an open-source testbed for simulation-based verification, AI-agent assessment, digital identity research, human-AI validation, and sovereign coordination infrastructure.

3.2 What OpenSpeX is

OpenSpeX is:

- An opportunity network powered by proven performance.
- A coordination and reputation layer evolving from SpaceXpanse.
- A verified-outcome infrastructure layer for humans, AI agents, services, validators, creators, simulations, and digital economies.
- A way to transform completed work into evidence, evidence into reputation, and reputation into access, rewards, trust, and future opportunity.
- A public category that makes the SpaceXpanse stack easier to understand beyond the older metaverse/GameFi framing.

3.3 What OpenSpeX is not

OpenSpeX should not be described as:

- A replacement for SpaceXpanse.
- A separate random token project.
- A claim that AI verification is solved.
- A fully decentralized network from day one.
- A finished digital civilization operating system.
- A promise of ROD price appreciation.
- A guaranteed earning system.
- An AWS, Roblox, OpenAI, Upwork, or LinkedIn killer.
- A giant 3D world launch.
- A universal verifier of all human and AI work.

The credible version is narrower and stronger: OpenSpeX starts with controlled environments, clear rules, captured evidence, human review where needed, reputation built from repeated validated outcomes, and progressive decentralization. [I-LAUNCH]

3.4 Public motto

The strongest motto stack is:

```
Prove. Build. Earn.

Prove ability.
Build trust.
Earn opportunity.
```

"Prove" anchors the evidence layer. "Build" covers reputation, tools, worlds, agents, tasks, assets, and trust. "Earn" includes opportunity, access, responsibility, recognition, settlement, and rewards, not only money. [I-MOTTO]

3.5 Internal category

The strongest internal category is **Verified Proof of Performance**: portable, verifiable evidence that a person, AI agent, service, team, model, validator, creator, or organization can consistently perform under defined conditions.

The public category should lead with opportunity because people do not primarily want proof as an end. They want jobs, clients, access, rewards, trusted roles, partnerships, rankings, funding, responsibility, and the chance to work on larger tasks. Proof is the engine. Reputation is the bridge. Opportunity is the destination. [I-POSITIONING]

---

4. Source basis and methodology

This whitepaper combines four source classes.

First, it uses internal SpaceXpanse and OpenSpeX materials, including the OpenSpeX Manifesto, OpenSpeX overview files, launch strategy files, MVP launch plans, SpaceXpanse evolution notes, ROD blockchain specs, SpaceXpanse documentation, simulator manuals, SpeXcode materials, RODNS and Nostr integration notes, ANGI/A2A/OASF/GraphRAG notes, O3DE/STORM simulation notes, and prior SpaceXpanse whitepaper work.

Second, it uses official SpaceXpanse public materials, including the ROD coin page, SpaceXpanse documentation, SpaceXpanse ROD technical documents, and SpaceXpanse Metaverse Simulator materials. These sources ground the claim that ROD provides blockchain, value/data storage, names, token/NFT, atomic trading, game/application interaction, mining, and settlement primitives. [E-SPACE-ROD] [E-SPACE-DOCS]

Third, it uses relevant external technical sources: Namecoin for the name-value and decentralized DNS lineage; Xaya XID for blockchain-name identity; Nostr for relay-based decentralized event transport; OpenID Connect for the identity-provider analogy; ERC-8004 for AI-agent identity/reputation/validation as an emerging design pattern; A2A for agent-to-agent communication; MCP for agent-tool interoperability; OASF for agent capability schema; GraphRAG for semantic knowledge-graph reasoning; O3DE for open high-fidelity simulation; and EU Digital Decade / AI Factories sources for institutional alignment. [E-NAMECOIN] [E-XID] [E-NOSTR] [E-OIDC] [E-ERC8004] [E-A2A] [E-MCP] [E-OASF] [E-GRAPHRAG] [E-O3DE] [E-EU-DD] [E-EU-AI]

Fourth, it uses current AI-evaluation context, including OpenAI commentary on benchmark decay and emerging benchmark research for real-world occupational and workflow tasks. [E-OPENAI-SWE] [E-OCCUBENCH] [E-JOBBENCH]

The methodology is conservative. The document distinguishes:

Status	Meaning
Current foundation	Existing documentation, codebase, conceptual base, or working primitive.

Status	Meaning
MVP	Near-term product loop that should be built and demonstrated first.
Planned	Plausible next-stage system that depends on MVP success.
Experimental	Research-heavy, high-risk, or not yet proven.
Vision	Long-term direction, not a present-tense claim.

---

5. Relationship between SpaceXpanse, OpenSpeX, ROD, and USR

5.1 SpaceXpanse is the foundation

SpaceXpanse began as a trustless GameFi blockchain metaverse platform for rapid metaverse creation, immersive user experience, and decentralized applications. That framing remains part of the history, but the stronger current interpretation is that SpaceXpanse is an open infrastructure stack for ownable digital economies. [I-EVOLUTION]

SpaceXpanse provides the deeper foundation:

- ROD blockchain.
- ROD name-value storage.
- SpeXID identity concepts.
- Tokens and NFTs.
- Atomic trading.
- RODPay and commerce primitives.
- SpeX Library and developer integration.
- DappEngine and creator tooling direction.
- Launchpad access layer.
- Metaverse Simulator and proof environments.
- Nostr/Metalog communication direction.
- SpeXcode creation layer.
- D.A.R.M.A. support and orchestration layer.

OpenSpeX does not discard this. It activates it.

5.2 OpenSpeX is the coordination and reputation layer

OpenSpeX is built from SpaceXpanse, not instead of it. Its role is to make the infrastructure useful for verified outcomes, operational reputation, AI-agent assessment, creator economies, human validation, microtasks, simulation credentials, and service coordination. [I-LAUNCH]

The clean relationship is:

```
SpaceXpanse = foundation stack
ROD = blockchain truth, naming, ownership, settlement, reputation anchor
SpeXID = identity and passport layer
SpeXcode = creation and deployment layer
USR = internal runtime/orchestration mechanism
OpenSpeX = public coordination, verification, reputation, opportunity layer
```

```
Launchpad = user/developer gateway
D.A.R.M.A. = operational assistant and support layer
```

5.3 ROD is the settlement and trust anchor

ROD should not be marketed as a speculative asset. It should be described by utility: fees, names, payments, rewards, identity anchoring, NFT/token transfers, marketplace settlement, reputation anchoring, service-provider registration, and challenge rewards. [I-LAUNCH]

Official SpaceXpanse material describes ROD as the native cryptocurrency and foundational blockchain/core wallet layer. It supports secure value/data storage, transaction processing, mining, network connectivity, fungible and nonfungible tokens, arbitrary names/digital IDs, data storage, atomic trading for in-game assets, and Play-to-Earn as a work-in-progress model. [E-SPACE-ROD]

5.4 USR is the internal runtime

USR, or Unified Service Runtime, should remain primarily technical. It is the orchestration and routing layer for tasks, service providers, SLAs, validation triggers, settlement, and reputation updates. Publicly, OpenSpeX should carry the simpler story: verified digital work, reputation, and opportunity. Technical documentation can describe OpenSpeX as powered by USR where appropriate. [I-LAUNCH]

5.5 The core loop

The mature SpaceXpanse/OpenSpeX loop is:

```
Create identity
-> Build or deploy task, world, asset, agent, service, or mission
-> Human or AI agent interacts with it
-> Evidence is captured
-> Result is verified
-> Reputation updates
-> Settlement or reward occurs in ROD or another configured rail
-> Identity and reputation are reused across the ecosystem
```

This is the new center. It is more durable than the term "metaverse." [I-EVOLUTION]

---

6. Design principles

6.1 Opportunity first

The user-facing promise should begin with opportunity, not protocol. OpenSpeX should speak to the outcome people care about: better tasks, trusted access, clients, rankings, rewards, partnerships, research credibility, agent selection, and future work.

6.2 Proof over claims

Claims remain useful, but claims should not be the final trust layer. OpenSpeX prioritizes evidence from completed actions, validation, telemetry, review, and repeat performance.

6.3 Reputation from repeated outcomes

One proof is not enough. Trust compounds through repeated validated outcomes across task types, environments, validators, and time. A provider with a large verified completion history has an advantage that cannot be copied overnight. [I-LAUNCH]

6.4 Settlement separated from experience

High-frequency gameplay, simulation, AI actions, and user interactions should not all occur directly on-chain. ROD should anchor compact commitments, names, settlement, state roots, reputation checkpoints, and ownership records. Heavy telemetry, event streams, social data, validation logs, and metadata should remain off-chain with verifiable pointers. [I-MVP]

6.5 Progressive decentralization

OpenSpeX should not claim full decentralization from day one. The credible path is:

```
Single proof loop
-> signed relay validation
-> public archives and recomputation
-> multiple relays
-> validator reputation
-> audits and disputes
-> higher-stakes verification tiers
-> eventual decentralized governance where proven
```

6.6 User sovereignty

Users should move toward owning identity, assets, reputation records, and settlement endpoints rather than renting them from platforms. This does not mean every user must self-custody from the first interaction. For onboarding, testnet, custodial, or walletless first actions may be appropriate. But the long-term direction should be user-owned identity and portable proof.

6.7 Open-source composability

The stack should be modular, inspectable, forkable where appropriate, and useful to developers. SpaceXpanse already emphasizes open-source and community-driven foundations. OpenSpeX should strengthen that by publishing schemas, examples, SDKs, regtest environments, reference relays, task definitions, and public status pages.

6.8 Controlled proof before broad claims

OpenSpeX should start where verification is realistic: tasks with explicit rules, telemetry, deterministic scoring, hidden test cases, reproducible outputs, or human-review protocols. Subjective, high-stakes, regulated, or safety-critical claims should be delayed until proper validation, audits, disputes, and governance exist. [I-LAUNCH]

---

7. System architecture

7.1 Layer map

```
User / human / AI agent / creator / validator / service provider

Launchpad / Web Dashboard / D.A.R.M.A.

Proof environments:
Void Runner Arena | Orbit Challenge | Simulator | O3DE Runtime | Microtasks | AI Runtime | DApps

ANGI / MCP / A2A / OASF / GraphRAG integration layer for agents

OpenSpeX coordination and reputation layer

USR runtime / routing / validation / settlement trigger layer

SpeXcode / DappEngine / SpeX Library creation and developer layer

RODNS / .spex / .rod discovery layer

Nostr / Metalog event distribution and off-chain metadata layer

ROD blockchain:
names | identity anchors | ownership | tokens/NFTs | commitments | reputation checkpoints | settlement
```

7.2 ROD blockchain layer

ROD is the base truth and settlement layer. The ROD technical specs describe SpaceXpanse as based on Namecoin, inheriting much of the Bitcoin/Namecoin model and adding a name-value database. They list ROD technical parameters including neoscrypt-xaya and SHA-256d merged mining, ChainID 1899, 30-second block time, confirmation parameters, name sizes, value sizes, and application/game interaction models. [I-ROD-SPECS]

ROD should anchor:

- Identity/name ownership.
- Payment and reward settlement.
- Digital asset ownership.
- Token/NFT records where supported.
- Atomic trading commitments.
- Proof roots.
- Reputation checkpoints.
- Validator/service-provider registrations.
- Discovery records.
- Compact task and leaderboard commitments.

ROD should not carry:

- Full telemetry streams.

- High-frequency game inputs.
- Raw AI prompts/outputs where privacy or size matters.
- Full social timelines.
- Large datasets.
- Long validation logs.
- Full simulation traces.

7.3 Namecoin and Xaya lineage

SpaceXpanse inherits important ideas from Namecoin and Xaya. Namecoin pioneered Bitcoin-derived blockchain naming, key/value records, decentralized DNS, and merged mining. [E-NAMECOIN] Xaya extended blockchain naming and game-state ideas into serverless games, game-state processors, and identity patterns. Xaya XID shows how blockchain names can become passwordless identity primitives where the owner of a name controls the associated metadata and signing authority. [E-XID]

OpenSpeX should use this lineage without overclaiming equivalence. It can adapt the pattern: a ROD-controlled name can become a persistent identity anchor, service record, relay pointer, agent passport, proof pointer, or payment endpoint.

7.4 SpeXID identity layer

SpeXID should be the identity/passport layer for humans, AI agents, validators, creators, service providers, organizations, worlds, and applications. Its functional analogy is "OpenID without a centralized identity provider." OpenID Connect lets clients verify a user's identity based on authentication by an authorization server and obtain profile claims. [E-OIDC] SpeXID can adapt this pattern so ROD name ownership, signatures, and metadata replace the centralized identity provider.

A minimal SpeXID profile may include:

```
{
 "schema": "spexid.profile.v0.1",
 "id": "alice.spex",
 "rodName": "id/alice",
 "entityType": "human|agent|validator|creator|provider|organization|world",
 "displayName": "Alice",
 "pubkeys": {
  "rod": "...",
  "nostr": "...",
  "agent": "..."
 },
 "capabilities": ["pilot", "validator", "ocr", "browser_task"],
 "payment": {
  "rod": "R..."
 },
 "reputation": {
  "checkpoint": "proof/alice/latest",
  "profileURI": "..."
 },
```

```
"status": "experimental"
}
```

7.5 OpenSpeX coordination layer

OpenSpeX coordinates tasks, evidence, validation, reputation, and settlement. It should not attempt to be every layer. Its boundaries should remain clear:

- It does not replace ROD consensus.
- It does not store all evidence on-chain.
- It does not replace Nostr for message transport.
- It does not replace MCP for tool invocation.
- It does not replace A2A for agent collaboration.
- It does not replace OASF for agent capability schemas.
- It does not replace GraphRAG for knowledge reasoning.
- It does not replace O3DE or the Metaverse Simulator for simulation execution.

OpenSpeX consumes outputs from these layers and turns them into proof, reputation, and settlement.

7.6 USR runtime layer

USR provides the internal runtime for routing tasks, selecting providers, enforcing SLAs where possible, triggering validation, updating reputation, and initiating settlement. It should be framed as coordination infrastructure rather than a universal verification oracle. Verification remains domain-specific.

7.7 Launchpad and dashboard

Launchpad should become the user gateway. Internal materials describe it as evolving into a hub for wallet, SpeXID, games, simulator, challenges, leaderboard, assets, marketplace, tasks, messages, D.A.R.M.A., and creator tools. [I-LAUNCH]

Because the legacy simulator/Launchpad roots are heavy, the near-term path should include a modern lightweight web dashboard before the full desktop Launchpad is mature. The dashboard should support:

- Sign-in / handle creation.
- SpeXID or temporary identity.
- Task browsing.
- Void Runner Arena launch.
- Orbit Challenge status.
- Validator UI.
- Reputation profile.
- Reward/badge display.
- ROD transaction or testnet proof display.
- Status labels for current, experimental, and roadmap components.

7.8 D.A.R.M.A. operational assistant

D.A.R.M.A. should make the ecosystem usable, not act as magical AI. Its roles include:

- Onboarding guide.
- Task explainer.
- Validator assistant.
- Documentation assistant.
- Reputation report generator.
- Launch-content generator.
- Agent orchestration helper.
- User support.
- Product and research synthesis assistant.

7.9 O3DE and high-fidelity simulation

The existing Metaverse Simulator provides orbital and physics heritage. O3DE can add a modern high-fidelity 3D, robotics, digital twin, and simulation runtime. O3DE is an open-source, real-time, multi-platform 3D engine for AAA games, cinema-quality worlds, and high-fidelity simulations under Apache-2.0. [E-O3DE]

The recommended architecture is not to replace the current simulator immediately, but to wrap and extend it:

```
Metaverse Simulator = orbital/physics heritage and scenario logic
O3DE Runtime = high-fidelity graphics, robotics, digital twins, ROS2-style simulation
OpenSpeX = task coordination, validation, reputation
ROD = final anchor and settlement
SpeXcode = mission/environment generation
```

This matters for institutional use cases: robotics, autonomy, digital twins, university labs, AI-agent control benchmarks, simulation-based credentials, and verified mission telemetry. [I-O3DE-STORM]

---

8. How OpenSpeX works

8.1 The universal outcome loop

OpenSpeX should use one simple loop across many domains:

```
Define task
-> Define success criteria
-> Capture evidence
-> Verify result
-> Score performance
-> Update reputation
-> Settle payment, reward, badge, or access
-> Make the proof reusable
```

This loop applies to a game run, a spaceflight mission, an AI-output review, an OCR extraction, a bug reproduction, a browser automation workflow, a creator asset validation, a translation check, a simulation challenge, or a service-provider SLA. [I-LAUNCH]

8.2 Participants

OpenSpeX supports several participant classes:

Participant	Role
Human performer	Completes tasks, missions, reviews, challenges, or services.
AI agent	Completes tasks, controls environments, produces outputs, collaborates with other agents.
Validator	Checks outputs, reviews ambiguity, audits results, validates telemetry.
Creator	Builds tasks, scenarios, worlds, assets, templates, workflows, and proof environments.
Service provider	Delivers outputs under task definitions or SLAs.
Business	Defines real tasks, buys verified outcomes, evaluates performers.
Researcher	Designs studies, evaluates agents, studies reputation, publishes datasets.
Institution	Runs education, certification, public-interest infrastructure, or simulation pilots.
Relay/operator	Provides event transport, indexing, validation services, or archives.

8.3 Task definition

A task should define:

- Task ID.
- Creator/issuer.
- Task type.
- Environment.
- Ruleset version.
- Success criteria.
- Evidence requirements.
- Validation method.
- Reward/fee model.
- Time limits.
- Risk level.
- Privacy level.
- Dispute rules.
- Status: experimental/MVP/verified.

Example:

```
{
  "schema": "openspex.task.v0.1",
  "taskID": "orbit-basic-0001",
  "issuer": "orbit.spex",
```

```
  "environment": "spacexpanse-simulator",
  "ruleset": "orbit-basic-v1",
  "goal": "reach stable low orbit under fixed constraints",
  "evidence": ["telemetryHash", "stateVectorSummary", "fuelRemaining", "trajectoryRoot"],
  "validation": "tier2",
  "reward": {"type": "badge+testnet-rod"},
  "status": "mvp"
}
```

8.4 Evidence capture

Evidence depends on domain. Examples:

Domain	Evidence
Void Runner Arena	Score, survival time, event hash, telemetry hash, client build, relay challenge, validation receipt.
Orbit Challenge	State vectors, altitude, velocity, fuel, trajectory summary, orbital stability window, telemetry hash.
OCR task	Input document hash, extracted text/structure, confidence, error report, validator feedback.
AI-output review	Prompt hash, output hash, rubric, validator scores, hallucination flags, corrections.
Browser task	Action log, page state references, screenshots where needed, completion proof, risk confirmations.
Creator asset review	Asset hash, metadata, license, quality checks, user/validator reviews.
Service SLA	Request, output, latency, availability, proof of delivery, validation result.

8.5 Verification

Verification must be domain-specific. OpenSpeX should not claim universal trustless verification. Instead, it should publish verification confidence levels:

Tier	Use case	Verification method
Tier 3	Low-stakes tasks	Hashes, logs, basic checks, simple consistency checks.
Tier 2	Medium-stakes tasks	Spot checks, human review, re-execution samples, server-side consistency.
Tier 1	High-stakes tasks	Validator consensus, audit trails, dispute mechanisms, independent test cases, stronger cryptographic and procedural controls.

The initial OpenSpeX MVP should focus on Tier 3 and Tier 2 tasks. High-stakes Tier 1 verification should remain roadmap until the validator, dispute, audit, and governance systems are proven. [I-LAUNCH]

8.6 Reputation update

A verified result produces a reputation delta. Reputation should be contextual, not a single universal score. A participant may be excellent at browser data extraction but weak at spacecraft control. An AI agent may be fast but unreliable. A validator may be accurate in OCR tasks but not in legal review.

A reputation update should include:

- Subject identity.
- Task type.
- Environment.
- Score.
- Validation confidence.
- Validator identity or method.
- Evidence root.
- Timestamp/block reference.
- Dispute status.
- Cost/time.
- Failure or success mode.

8.7 Settlement

Settlement can take several forms:

- ROD payment.
- Testnet reward.
- Badge.
- Reputation increase.
- Access to harder tasks.
- Leaderboard position.
- Creator royalty.
- Validator reward.
- Business purchase of verified output.
- Dataset licensing or access.

Not every task needs immediate token payment. In many early environments, badges, leaderboards, testnet records, or reputation may be safer. ROD should be utility-framed, not investment-framed.

8.8 Opportunity creation

Opportunity emerges when proof becomes useful outside the original task. Examples:

- A validator with a reliable history gets invited to higher-value reviews.
- An AI agent with strong browser-task performance gets selected for business pilots.
- A human pilot with strong Orbit Challenge records earns simulation credentials.
- A creator whose tasks generate useful data earns fees or reputation.
- A business identifies the best performer based on validated results.
- A student uses verified mission completions as evidence of skill.
- A service provider earns trust by repeatedly meeting objective criteria.

This is why OpenSpeX is not merely a proof network. It is an opportunity network powered by proof.

---

9. Identity, SpeXID, and Agent Passports

9.1 Identity should be provable

The OpenSpeX Manifesto states that identity, ownership, work, skill, contribution, performance, and reputation should be provable. [I-MANIFESTO] SpeXID is the identity layer that makes this operational.

A SpeXID does not need to reveal a real-world legal identity by default. It should prove control of a persistent digital identity and attach relevant claims, capabilities, reputation, and endpoints. Depending on task type, stronger verification may be required.

9.2 Human profiles

A human profile may include:

- Handle.
- ROD name.
- Display name.
- Public keys.
- Wallet endpoint.
- Nostr/Metalog endpoint.
- Completed tasks.
- Reputation categories.
- Validator history.
- Creator contributions.
- Privacy preferences.
- Optional credentials.

9.3 AI-agent profiles

AI agents need portable identity and operational reputation. ERC-8004 is useful as an external design pattern because it frames AI-agent interoperability around three registries: Identity, Reputation, and Validation. [E-ERC8004] OpenSpeX should not copy it literally unless technically and legally appropriate, but it can learn from the structure.

An OpenSpeX Agent Passport may include:

```
{
 "schema": "openspex.agent_passport.v0.1",
 "id": "navigator.agent.spex",
 "owner": "alice.spex",
 "modelFamily": "declared-or-hidden",
 "capabilities": ["browser_navigation", "ocr", "simulation_control"],
 "tools": ["mcp", "playwright", "a2a"],
```

```
 "riskPolicy": "requires-human-confirmation-for-payments",
 "reputation": {
  "tasksCompleted": 0,
  "successRate": null,
  "domains": []
 },
 "validationLevel": "experimental",
 "paymentEndpoint": "pay/navigator",
 "status": "mvp"
}
```


9.4 Validators

Validators should have their own reputation. A validator is not trusted merely because they sign a result. Their history should include:

- Validation categories.
- Agreement with consensus or gold-standard tasks.
- Dispute rate.
- Reversal rate.
- Latency.
- Specialization.
- Conflict-of-interest disclosures where applicable.
- Stake or bond, if later implemented.

9.5 Organizations and institutions

Organizations can issue tasks, sponsor challenges, operate relays, run academic pilots, or publish validation schemas. Their profiles should distinguish between identity proof, technical authority, legal authority, and ecosystem reputation.

---

10. ROD as settlement and trust anchor

10.1 Utility role

ROD is the utility and settlement asset that anchors names, value transfer, digital ownership, rewards, and reputation events inside the SpaceXpanse/OpenSpeX stack. It should be presented as infrastructure, not an investment story. [I-LAUNCH]

ROD can support:

- Fees for name registration and updates.
- Payment for tasks.
- Rewards for challenges.
- Validator compensation.
- Creator royalties.
- Digital asset trades.
- Atomic trading.

- Reputation checkpoint anchoring.
- Service-provider registration.
- Relay or endpoint discovery.

10.2 Technical parameters snapshot

The ROD technical specs include the following public parameters. Live network values should always be verified through current node/explorer data.

Parameter	Value / description
Name	SpaceXpanse
Ticker	ROD
Address prefix	R
P2P port	11998
RPC port	11999
PoW algorithms	neoscrypt-xaya and SHA-256d merged mining
ChainID	1899
Block time	30 seconds
Spend confirmations	6
Block confirmations	120
Name size	Up to 256 bytes
Value size	Up to 2,048 bytes
Initial / main reward parameters	See ROD technical specs
Halving schedule	Every 1,054,080 blocks for initial period, per specs

[I-ROD-SPECS]

10.3 Compact commitments

Because the ROD name-value layer is suited to small critical records, OpenSpeX should store compact commitments on-chain and full evidence off-chain. [I-MVP]

On-chain:

- Round commitment.
- Leaderboard root.
- Validation root.
- Top-score summary.
- Relay signature.
- Reputation checkpoint.
- Proof pointer.

Off-chain:

- Full telemetry.
- Event stream.
- Raw session data.
- Debug payload.
- Validation logs.
- Screenshots or video evidence.
- Large metadata.

10.4 ROD and trust boundaries

ROD provides final anchoring. It does not magically prove all claims. A false relay could anchor a false commitment. A manipulated client could submit a dishonest claim. A validator could collude. The system must therefore separate:

```
Client claim = evidence submitted by untrusted environment
Relay validation = first authoritative check
Independent recomputation = auditability
ROD anchor = final timestamped commitment to the validation state
```

This distinction is critical for credibility.

---

11. RODNS, .rod, .spex, Nostr, and discovery

11.1 ROD as authority, Nostr as transport

The clean pattern is:

```
ROD = authority / discovery / routing key
Nostr = transport / delivery / messaging layer
```

ROD should tell applications who controls a name, which keys are valid, which relays or endpoints are recommended, and which records are authoritative. Nostr and Metalog-style relays can distribute messages, social activity, event receipts, proof pointers, result claims, comments, reactions, service announcements, and task updates. [I-ROD-NOSTR]

Nostr itself is an open protocol based on signed events and relays, designed without a trusted central server. [E-NOSTR]

11.2 .rod and .spex split

The strongest namespace split is:

```
.rod = chain-native / coin-native / protocol namespace
.spex = ecosystem-facing identity, service, app, agent, proof, and reputation namespace
```

Use `.spex` for public-facing discovery:

```
alice.spex
agent.alice.spex
voidrunner.spex
orbit.spex
market.spex
relay.spex
validator42.spex
```

Use `.rod` for deeper coin-native or infrastructure-native endpoints:

```
pay.alice.rod
node42.rod
pool.rod
wallet.rod
```

The principle: ROD stores the record; `.spex` is what humans and apps resolve. [I-RODNS]

11.3 Namespace model

Recommended namespace families:

```
spex/   public .spex names
id/     SpeXID identity records
agent/  AI agent records
svc/    service/provider records
relay/  Nostr/Metalog relay records
app/    DApp/world/proof-environment records
pay/    RODPay/payment records
proof/  OpenSpeX verification/reputation records
rod/    coin-native infrastructure records
```

Example:

```
spex/alice       -> alice.spex
agent/darma      -> darma.agent.spex
app/voidrunner   -> voidrunner.spex
svc/validator42  -> validator42.spex
pay/alice        -> pay.alice.spex
relay/main       -> main.relay.spex
```

11.4 Relay identity

A relay should not be trusted because of its domain. It should be trusted because a ROD-controlled record points to a relay public key and that relay signs its events. The domain is only a replaceable transport endpoint. [I-RODNS]

Example relay record:

```
{
 "schema": "spex.relay.v0.1",
 "pubkey": "npub1...",
 "endpoints": ["wss://relay1.example", "wss://relay2.example"],
 "role": ["feed", "openspex", "agent", "validator"],
 "owner": "validator.spex",
 "capabilities": {
  "payments": true,
  "validation": true,
  "proofPointers": true
 }
}
```

11.5 RODNS implementation path

A clean implementation sequence:

1. Build or adapt `roddnsd` as a separate resolver daemon.

2. Resolve `.rod` from ROD name records.

3. Add strict JSON schemas for records.

4. Add JSON API before full browser integration.

5. Add SpeX Library resolver methods.
6. Add Launchpad name manager.
7. Build `.spex` resolver using ROD name records plus Nostr/SpeX metadata.
8. Add relay, agent, validator, payment, and proof discovery.
9. Add signatures, delegation, trust labels, and phishing warnings.

This keeps ROD Core as the truth layer while moving product logic into resolvers and libraries. [I-RODNS]

---

12. Proof environments

12.1 Proof environments are not the product

Void Runner Arena, Orbit Challenge, Battleships, simulator missions, OCR tasks, AI-output review tasks, and browser workflows are proof environments. They are not the whole product.

The product is the common system that creates, runs, verifies, records, compares, and monetizes performance across any environment where results can be measured. [I-OVERVIEW]

12.2 Void Runner Arena

Void Runner Arena is the first fast proof environment. It is browser-native, low-friction, public, measurable, and suitable for human-vs-AI comparison.

Strategic role:

- Fast onboarding.
- Public leaderboard.
- Registered handles.
- Score submission.
- Telemetry/evidence loop.
- Chain-backed or ROD-anchored results.
- Human-vs-AI arena.
- First OpenSpeX reputation loop.

Conceptual flow:

```
Void Runner Arena
-> execution environment
-> telemetry / evidence
-> relay validation
-> leaderboard / reputation
-> ROD settlement or checkpoint
-> OpenSpeX profile
```

[I-VOID]

The MVP should not add gameplay complexity. It should prove one verified-outcome transaction loop with real users. Browser telemetry must be treated as a claim, not truth. [I-MVP]

12.3 Orbit Challenge

Orbit Challenge is the serious simulation proof environment. It should use the Metaverse Simulator and later O3DE/ROS2 extensions for physics-based tasks, mission planning, telemetry, and objective evaluation.

First mission:

```
Reach stable orbit under fixed rules.
```

Evidence:

- Altitude.
- Velocity.
- Fuel.
- Trajectory.
- Duration.
- Orbital stability.
- State vectors.
- Telemetry hash.

Output:

- Verified success/failure.
- Leaderboard.
- Reputation badge.
- Optional ROD/testnet reward.
- Public mission report.

Orbit Challenge is important because it bridges from game-like proof to institutionally credible simulation. It can appeal to students, universities, STEM educators, aerospace enthusiasts, digital-twin researchers, AI-agent testers, and public-sector pilots. [I-LAUNCH]

12.4 Metaverse Simulator

The SpaceXpanse Metaverse Simulator is based on Newtonian mechanics and uses a solar-system playground with spacecraft, celestial bodies, mission planning, and community add-ons. [E-SPACE-DOCS] It is especially suitable for:

- Orbital mechanics education.
- Mission training.
- Scenario creation.
- Scientific visualization.
- Engineering prototyping.
- Simulation-based proof tasks.
- Blockchain-backed achievements.

12.5 O3DE Runtime and ROS2 mission sandbox

O3DE can become the modern visual and robotics simulation layer for OpenSpeX. The first serious version could be an O3DE ROS2 Mission Sandbox where humans or AI agents control a spacecraft, rover, drone, or robot through a simulation, complete a measurable mission, generate telemetry, receive a score, and build reputation. [I-O3DE-STORM]

12.6 Battleships and simpler game environments

Battleships and similar turn-based games can provide deterministic, lower-frequency proof environments. They are useful for testing:

- Move validation.
- Game-state processing.
- Player identity.
- Atomic rewards.
- Leaderboards.
- NFT or asset influence where appropriate.
- Simple player-vs-player reputation.

12.7 Document Extraction Arena

An OCR or document extraction arena can become one of the most practical enterprise proof environments. A task issuer provides messy documents, PDFs, tables, or scans. Humans, OCR engines, or AI agents extract text and structure. Validators compare outputs against a rubric or gold standard. The result creates:

- A business answer.
- AI training/evaluation data.
- Performer reputation.
- Validator reputation.
- Task template for future use.

This is a strong bridge from games/simulations into real work.

12.8 Human microtasks

Microtasks are not the final vision, but they may bootstrap the first economically active coordination market. Useful microtasks include:

- AI-output review.
- Hallucination checks.
- Citation validation.
- Mission testing.
- Bug reproduction.
- Scenario review.
- Asset tagging.
- Gameplay balancing.
- Simulator training tasks.
- AI-agent grading.
- Translation/localization.
- Documentation review.

This should be framed as human-in-the-loop verification infrastructure, not generic gig work. [I-LAUNCH]

12.9 AI Runtime and service coordination

AI Runtime can demonstrate machine-service coordination: task routing, output hashing, latency measurement, provider reputation, validation, and settlement. The first commercial examples should be practical:

- AI-output review.
- Document summarization verification.
- Code-agent evaluation.
- Workflow-agent reliability scoring.
- Browser automation tasks.

Do not publicly claim "decentralized AI verification solved." Say AI Runtime is a controlled PoC for OpenSpeX service coordination and reputation. [I-LAUNCH]

---

13. SpeXcode and the creation layer

13.1 SpeXcode is not merely a coding assistant

SpeXcode should be the AI-powered creation and deployment layer for SpaceXpanse/OpenSpeX. It can turn prompts and specifications into tasks, scenarios, workflows, assets, missions, worlds, simulations, and autonomous environments. [I-SPEXCODE]

Better framing:

SpeXcode is the deployment cockpit for executable digital economies.

13.2 Why SpeXcode matters

OpenSpeX needs continuous task generation. Static benchmarks decay. Static games become solved. Static datasets become contaminated. SpeXcode can generate new task variants, hidden edge cases, mission designs, browser workflows, simulation scenarios, AI-agent challenges, creator tasks, procedural worlds, and validation puzzles. [I-SPEXCODE]

This gives OpenSpeX a potential long-term advantage: dynamic proving environments.

13.3 SpeXcode loop

```
Creator has idea
-> SpeXcode helps generate task, mission, world, asset, or workflow
-> DappEngine / SpeX Library / developer tools assemble it
-> Launchpad publishes it
-> Humans or AI agents execute
-> OpenSpeX captures evidence
-> USR verifies outcome
-> Reputation updates
-> ROD settles rewards or checkpoints
```

13.4 SpeXcode outputs

SpeXcode can generate:

- Simulator scenarios.
- Lua scripts.
- Planets and solar systems.
- Spacecraft, stations, meshes, and assets.
- Game rules and scoring systems.
- AI NPC behavior.
- Challenge templates.
- Microtask templates.
- Browser workflows.
- Document processing tasks.
- O3DE missions and scenes.
- Agent evaluation tasks.
- Validation rubrics.
- Reputation report templates.

13.5 Creator economy

SpeXcode helps move the ecosystem from consuming worlds to building worlds. Creators can publish proof environments, templates, missions, assets, and workflows. If those environments produce useful results, creators can earn fees, reputation, royalties, or visibility.

---

14. ANGI and agent-native environments

14.1 Why agents need a native interface

Today, AI agents often operate software by looking at screenshots, parsing human-facing pages, and sending keyboard or mouse events. This works, but it is inefficient and brittle. A screenshot is a poor interface for an autonomous system.

A future-ready environment should expose a machine-readable description of:

```
state
available actions
required inputs
constraints
risk
permissions
transitions
evidence hooks
expected result
```

This is the purpose of ANGI: Agent Native Graph Interface. [I-ANGI]

14.2 ANGI definition

ANGI is a machine-readable interface that allows autonomous agents to understand, reason about, and operate environments through structured states, actions, constraints, semantics, and evidence rather than relying primarily on visual interpretation. [I-ANGI]

Useful analogy:

```
HTML = human-native interface
API = software-native interface
ANGI = agent-native interface
```

14.3 ANGI boundaries

ANGI should stay narrow. It should not become the identity standard, payment protocol, reputation protocol, message protocol, tool protocol, or knowledge graph. It should define the environment contract.

ANGI plugs into:

- OASF for agent capability descriptions.
- GraphRAG for semantic context.
- MCP/OpenAPI for tool execution.
- A2A for agent collaboration.
- OpenSpeX for verification and reputation.
- ROD for anchoring.

14.4 OASF, A2A, MCP, and GraphRAG alignment

OASF provides standardized schemas for agent capabilities, interactions, metadata, and relationships across distributed systems. [E-OASF] MCP is an open standard for connecting AI applications to external systems, tools, data, and workflows. [E-MCP] A2A provides patterns for agent discovery, task lifecycle, messages, artifacts, and agent-to-agent coordination. [E-A2A] GraphRAG extracts knowledge graphs and supports richer retrieval/reasoning over complex context. [E-GRAPHRAG]

The clean architecture:

```
RODNS / .spex = discovery and identity root
OASF = agent capability description
ANGI = environment state/action interface
GraphRAG = environment and user knowledge reasoning layer
MCP / APIs / controllers = execution layer
A2A = agent-to-agent coordination
OpenSpeX = evidence, validation, reputation
ROD = final commitment and settlement
```

[I-ANGI]

14.5 ANGI in Void Runner Arena

For Void Runner Arena:

```
ANGI-VR = ship state, asteroid vectors, powerups, scoring, allowed controls
```

```
OASF Agent Profile = pilot agent, visual/control capability, risk policy
GraphRAG Context Pack = rules, scoring logic, past runs, leaderboard economy
OpenSpeX = run claim, validation receipt, reputation delta, ROD anchor
```

14.6 ANGI for websites and digital services

For websites:

```
ANGI-Web = page state, actions, forms, risks, transitions
OASF = browser agent, validator agent, auditor agent
GraphRAG = site policy, product catalog, FAQ, legal rules, user preferences
OpenSpeX = evidence logs and completed task records
```

This points toward a future where serious services expose an AI-facing layer just as they expose a web-facing layer today.

---

15. Reputation model

15.1 Reputation should be earned from evidence

Reputation should not be a popularity score. It should be an accumulated record of verified performance. OpenSpeX reputation should capture what a participant did, under what rules, with what evidence, how it was validated, what confidence level applies, and whether disputes occurred.

15.2 Contextual reputation

A single global score is too crude. OpenSpeX should use contextual profiles:

- Pilot / control reputation.
- Browser automation reputation.
- OCR / document extraction reputation.
- Validator reputation.
- Creator reputation.
- Service-provider reputation.
- Simulation mission reputation.
- AI-agent reliability reputation.
- Dispute handling reputation.
- Cost-performance reputation.

15.3 Reputation event

Minimal reputation event:

```json
{
 "schema": "openspex.reputation_delta.v0.1",
 "subject": "alice.spex",
 "taskID": "voidrunner-round-0001",
 "environment": "voidrunner-arena",
 "category": "pilot_control",
 "score": 84210,
```

```
  "delta": 12,
  "validationTier": "tier3",
  "validationHash": "sha256:...",
  "evidenceRoot": "sha256:...",
  "validator": "relay/voidrunner-main",
  "status": "valid",
  "createdAtBlock": 123456
}
```


15.4 Reputation decay and freshness

Some reputation should decay. A 2024 browser automation record may not prove 2026 ability. A validator may become less reliable. An AI agent may change model versions. OpenSpeX should store time and version metadata, not only aggregate points.

15.5 Failure records

Failure should not be hidden by default. Useful reputation includes:

- Failed attempts.
- Abandoned tasks.
- Dispute outcomes.
- Hallucination flags.
- Invalid telemetry.
- Rejected outputs.
- Timeouts.
- Over-budget completions.

A high-trust system cannot only record successes.

15.6 Portability and privacy

Reputation should be portable, but not every detail should be public. OpenSpeX should support:

- Public summaries.
- Private evidence with public commitments.
- Selective disclosure.
- Task-level privacy flags.
- Enterprise private pilots with optional public reputation claims.
- Zero-knowledge or privacy-preserving methods as later research.

---

16. Verification model and evidence tiers

16.1 Client claims are not truth

The browser client, game client, or agent runtime is untrusted. It may be manipulated. It may crash. It may lie. It may expose debug state. It may be controlled by a malicious user.

Correct posture:

```
Client payload = claimed run evidence
Relay validation = verified result, subject to its own trust level
Independent recomputation = audit path
ROD record = compact final commitment
```

[I-MVP]

16.2 Run claim schema

Void Runner Arena example:

```json
{
 "protocol": "openspex.voidrunner.run_claim.v1",
 "sessionID": "uuid",
 "handle": "player-handle",
 "roundID": "vr-000001",
 "timestampStart": 0,
 "timestampEnd": 0,
 "survivalTimeMs": 0,
 "bonusTimeMs": 0,
 "score": 0,
 "telemetryHash": "sha256",
 "clientBuild": "voidrunner-build",
 "clientSignature": "signature"
}
```

[I-MVP]

16.3 Validation receipt schema

```json
{
 "protocol": "openspex.voidrunner.validation_receipt.v1",
 "sessionID": "uuid",
 "roundID": "vr-000001",
 "handle": "player-handle",
 "claimedScore": 0,
 "verifiedScore": 0,
 "valid": true,
 "auditFlags": [],
 "telemetryHash": "sha256",
 "validationHash": "sha256",
 "relayID": "relay-public-key",
 "relaySignature": "signature"
}
```

[I-MVP]

16.4 Reputation delta schema

```json
{
 "protocol": "openspex.reputation_delta.v1",
 "subject": "handle-or-spexid",
 "source": "voidrunner-arena",
```

```
  "roundID": "vr-000001",
  "sessionID": "uuid",
  "delta": 0,
  "score": 0,
  "reason": "verified_run",
  "validationHash": "sha256",
  "createdAt": 0
}
```

[I-MVP]

16.5 Round commitment

A ROD-anchored round commitment might include:

```
{
  "schema": "openspex.round_commit.v0.1",
  "app": "voidrunner-arena",
  "roundID": "vr-000001",
  "ruleset": "voidrunner-rules-v1",
  "receiptRoot": "sha256:...",
  "leaderboardRoot": "sha256:...",
  "reputationRoot": "sha256:...",
  "top": [
    {"subject": "alice.spex", "score": 84210},
    {"subject": "agent42.spex", "score": 80100}
  ],
  "relay": "relay/voidrunner-main",
  "sig": "..."
}
```

The exact payload must fit the ROD value-size constraint. If it does not, store a hash and pointer, not the whole object.

16.6 Dispute model

OpenSpeX should support disputes from early design:

- Flag suspicious results.
- Delay rewards for suspicious accounts.
- Allow manual review.
- Maintain dispute status in reputation.
- Use gold-standard tasks to assess validators.
- Use random audits.
- Apply validator reputation weighting.
- Avoid high-stakes automated slashing until proven.

16.7 Verification confidence

Every result should display confidence level, not merely "verified".

Examples:

Label	Meaning
Claimed	Submitted by client or agent, not yet checked.
Relay-verified	Accepted by a known relay under published rules.
Recomputed	Recomputed or checked against archived evidence.
Human-reviewed	Validated by human reviewer(s).
Consensus-validated	Accepted by multiple validators under a published protocol.
Disputed	Under challenge or partially invalidated.
Finalized	Anchored and outside normal dispute window.

17. Business model and market category

17.1 Market category

OpenSpeX should create a category rather than enter weak existing buckets. The best category is:

Verified Digital Economy Infrastructure

Definition:

A verified digital economy infrastructure layer coordinates people, AI agents, services, tasks, evidence, reputation, and settlement across interoperable digital ecosystems. [I-LAUNCH]

17.2 Why not existing categories

Category	Why it is insufficient
Metaverse	Carries baggage from empty worlds, land speculation, and VR hype.
GameFi	Too narrow and often associated with token incentives before utility.
AI benchmark	Too narrow; OpenSpeX includes humans, services, creators, validators, simulations.
Cloud marketplace	Cloud sells capacity; OpenSpeX sells verified outcomes and reputation.
Freelance platform	Freelance platforms own reputation; OpenSpeX makes performance portable.
Blockchain ecosystem	Too generic; OpenSpeX focuses on coordination and proof environments.
Creator platform	Creator platforms host content; OpenSpeX links creation to proof, reputation, and settlement.

17.3 Product: verified performance intelligence

The clearest commercial product is **verified performance intelligence**: practical knowledge of who or what can complete a task, how well they completed it, what evidence proves it, and how that proof can create trust, payment, reputation, and future opportunity. [I-OVERVIEW]

A completed task can create three assets at once:

1. A result a business can use.

2. Structured execution data an AI developer can learn from.

3. A reputation signal showing who or what can deliver under real conditions.

17.4 Revenue possibilities

Potential revenue models:

- Task posting fees.
- Verification fees.
- Enterprise sandbox design.
- Private pilot deployments.
- Reputation analytics dashboards.
- AI-agent reliability reports.
- Dataset licensing where legally and ethically appropriate.
- Creator fees for paid proof environments.
- Validator marketplace fees.
- RODPay/Democrit marketplace fees.
- Sponsored challenges.
- Institutional research partnerships.
- Simulation credential programs.
- API access for verified performance records.

These should be introduced carefully and only after product proof exists.

17.5 Enterprise wedge

Do not start enterprise conversations with full decentralization. Start with auditable task workflows:

- Document extraction.
- Customer support triage.
- Invoice checking.
- Lead research.
- Website testing.
- Compliance review.
- Data labeling.
- Translation QA.
- AI-output validation.
- Browser automation.

OpenSpeX can turn these into controlled tasks with evidence capture, hidden edge cases, scoring, human or automated verification, and performance records.

17.6 Institutional wedge

OpenSpeX aligns with EU and institutional priorities when framed as open-source, simulation-based, AI-evaluation, identity, skills, and digital-sovereignty infrastructure. The EU Digital Decade emphasizes a human-centric, sustainable digital transformation across digital skills, connectivity, business, and public services. [E-EU-DD] EU AI Factories combine computing power, data, talent, supercomputing centers, universities, SMEs, industry, and other actors to accelerate trustworthy AI ecosystems. [E-EU-AI]

OpenSpeX can align with:

- AI-agent assessment.
- Simulation-based credentials.
- Digital skills.
- Open-source infrastructure.
- Decentralized identity research.
- Human-in-the-loop validation.
- Digital twin and robotics simulation.
- Sovereign coordination infrastructure.

---

18. Use cases by stakeholder

18.1 Normal users

OpenSpeX lets digital effort follow the user. Useful actions can become part of a growing record of ability, reliability, and trust, opening better tasks, rankings, access, collaborations, rewards, and discovery. [I-OVERVIEW]

18.2 Freelancers and workers

OpenSpeX helps reduce platform reset. Instead of rebuilding credibility from zero across every marketplace, verified work can become a portable performance record.

18.3 Microtask workers

OpenSpeX can turn repeated small tasks into visible reliability signals: speed, accuracy, consistency, validation quality, and domain-specific skill.

18.4 Gamers and pilots

OpenSpeX can make skill count beyond one scoreboard. A verified run becomes part of a performance history, not merely a temporary score.

18.5 AI-agent builders

OpenSpeX gives agents a way to earn trust through real outcomes. Instead of claiming an agent is reliable, builders can let it prove performance across tasks, seeds, environments, validators, and versions.

18.6 Validators

Validators earn reputation by judging outputs accurately. Their work becomes more valuable when it is itself validated over time.

18.7 Businesses

Businesses can choose people, agents, and services based on checked results rather than vendor promises, demos, closed ratings, or generic benchmark claims.

18.8 Creators

Creators can build proof environments, tasks, assets, scenarios, workflows, missions, and worlds. If those environments produce useful verified performance, creators can earn reputation, revenue, and ecosystem demand.

18.9 Developers

Developers can build digital economies without rebuilding identity, payments, ownership, communication, reputation, and verification from scratch. [I-LAUNCH]

18.10 Scientists and educators

OpenSpeX can turn simulations, missions, and repeatable tests into structured evidence. Students can complete missions and earn verifiable records. Researchers can compare agents or validation methods. Institutions can run open testbeds.

18.11 Investors and ecosystem partners

The investment thesis should be utility and infrastructure adoption, not token appreciation. The moat can become accumulated reputation, identity graph, proof environments, developer tooling, settlement layer, and institutional legitimacy. [I-LAUNCH]

18.12 Public sector and sovereignty stakeholders

OpenSpeX can be positioned as open infrastructure for identity, simulation, AI coordination, verified work, digital skills, and interoperable virtual economies. This should be framed as research and pilot infrastructure, not as a finished sovereign operating system.

---

19. Governance, openness, and decentralization path

19.1 Open-source as a trust lever

Open source matters because trust depends on inspectability. OpenSpeX should publish:

- Schemas.
- Reference clients.
- Reference relays.
- Validation logic.
- Task templates.
- Reputation algorithms.
- ROD anchoring formats.
- Threat models.

- Test vectors.
- Regtest instructions.
- Current/planned/experimental status.

19.2 Progressive governance

Governance should not be overbuilt before usage. Early governance can focus on:

- Schema review.
- Task standard approval.
- Validator guidelines.
- Dispute process.
- Security disclosures.
- Open-source contribution process.
- Roadmap transparency.

Formal DAO governance, staking/slashing, or high-stakes validator economics should be delayed until proof loops are stable.

19.3 Decentralization milestones

```
Stage 0: Single reference relay, public schemas, ROD testnet/mainnet anchors.
Stage 1: Multiple relays, archive mirrors, recomputable leaderboards.
Stage 2: Validator reputation and dispute process.
Stage 3: Task-specific validator sets and confidence levels.
Stage 4: Creator-operated environments and independent validators.
Stage 5: Broader decentralized governance for mature components.
```

19.4 Foundation and community

SpaceXpanse's open-source and foundation-operated ethos should remain visible. OpenSpeX should not hide its roots. The correct message is:

SpaceXpanse is not being abandoned. It is being opened. OpenSpeX is the broader infrastructure layer emerging from SpaceXpanse. [I-LAUNCH]

---

20. Security, privacy, and threat model

20.1 Threat: client tampering

A user can manipulate a browser client, game state, telemetry object, or debug payload. Mitigation:

- Treat client data as claims.
- Require relay-issued challenges.
- Hash canonical event streams.
- Validate against server-side or mathematical consistency.
- Use anomaly detection.
- Use replay audits.
- Anchor only validated commitments.

20.2 Threat: relay fraud

A relay may sign false results or omit evidence. Mitigation:

- Relay identity via RODNS and public keys.
- Public archives.
- Recomputable roots.
- Multiple relays.
- Relay reputation.
- Dispute process.
- Independent audits.

20.3 Threat: validator collusion

Validators may collude with participants. Mitigation:

- Random assignment.
- Validator reputation.
- Gold-standard tasks.
- Weighted consensus.
- Hidden control tasks.
- Delayed rewards for suspicious accounts.
- Dispute queues.

20.4 Threat: Sybil attacks

Participants may create many identities. Mitigation:

- Costed name creation or reputation bootstrap friction.
- Task-specific rate limits.
- Progressive trust levels.
- Identity age and proof history.
- Optional stronger verification for higher-stakes tasks.

20.5 Threat: benchmark contamination

Tasks may be leaked, memorized, or overfit. Mitigation:

- Dynamic task generation.
- SpeXcode-generated variants.
- Hidden test cases.
- Rotating validation criteria.
- Fresh seeds.
- Private enterprise task sets.
- Public but versioned benchmarks.

20.6 Threat: privacy leakage

Evidence can contain sensitive data. Mitigation:

- Minimize public evidence.

- Store hashes and commitments publicly.
- Keep raw data encrypted or access-controlled.
- Use private pilots.
- Separate public reputation from private evidence.
- Support deletion or retention policies where legally required, while preserving non-sensitive commitments.

20.7 Threat: overclaiming trustlessness

The biggest reputational risk is claiming more than the system proves. Mitigation:

- Use status labels.
- Publish limitations.
- Use verification tiers.
- Avoid "trustless AI verification solved."
- Avoid "fully decentralized from day one."
- Start with objective tasks.

---

21. Risks and mitigations

Risk	Why it matters	Mitigation
Too much vision, too little product	Users may dismiss the system as another speculative infrastructure story.	Lead with one working loop. Publish current/planned/experimental status.
Looks like another metaverse	"Metaverse" has market baggage.	Call games/simulators proof environments. Lead with verified performance.
Looks like token speculation	Institutional trust drops.	Frame ROD as utility and settlement. Avoid price language.
AI hype contamination	Users distrust magical claims.	Focus on measurable agent performance and failure records.
Verification complexity	Universal verification is impossible.	Start with controlled tasks and verification tiers.
Onboarding friction	Wallet-first flows can block users.	Browser-first, optional testnet/custodial first action, upgrade to SpeXID later.
Validator fraud	Bad validation corrupts reputation.	Validator reputation, audits, gold tasks, disputes.
Static benchmark decay	Agents may overfit tasks.	Dynamic task generation and hidden edge cases.
Privacy leakage	Evidence may contain sensitive material.	Commitments on-chain, raw evidence off-chain, access controls.
Ecosystem confusion	Community may think OpenSpeX replaces SpaceXpanse.	Repeat: OpenSpeX is built on SpaceXpanse and expands ROD utility.
Roadmap gravity	Too many future systems slow shipping.	Ruthless sequencing: prove one loop first.

22. Roadmap and execution doctrine

22.1 Execution doctrine

Stop explaining the whole universe. Prove one loop. Then another. Then connect them. [I-LAUNCH]

22.2 Phase 0: Reframe

Deliverables:

- OpenSpeX manifesto.
- Architecture diagram.
- SpaceXpanse -> OpenSpeX explainer.
- ROD utility map.
- Current/planned/experimental status page.
- Public vocabulary guide.

22.3 Phase 1: Lightweight proof

Lead with Void Runner Arena.

Deliverables:

- Public challenge page.
- Handle / temporary identity.
- Run claim schema.
- Validation receipt schema.
- Leaderboard root.
- Reputation delta.
- ROD/testnet anchor.
- Public profile.

22.4 Phase 2: Deep proof

Launch Orbit Challenge.

Deliverables:

- Basic mission rules.
- Telemetry capture.
- State-vector summary.
- Mission validation.
- Educational badge.
- Leaderboard.
- Reputation category.

22.5 Phase 3: Creator proof

Introduce SpeXcode-assisted mission/scenario generation.

Deliverables:

- Task template generator.
- Scenario generator.
- Validation rubric generator.
- Creator profile.
- Creator reward/reputation loop.

22.6 Phase 4: Work proof

Launch human-in-the-loop microtasks and AI-output review.

Deliverables:

- Task dashboard.
- Validator UI.
- AI-output review tasks.
- Document extraction arena.
- Human validator reputation.
- Business pilot workflow.

22.7 Phase 5: AI-agent proof

Launch AI-agent benchmark pilots.

Deliverables:

- Agent passport.
- OASF profile import/reference.
- ANGI profile for at least one environment.
- A2A/MCP pilot integrations.
- Agent reliability report.

22.8 Phase 6: Institutional infrastructure

Deliverables:

- Orbit Challenge Academic Edition.
- Open datasets where appropriate.
- Reproducibility protocol.
- University pilot package.
- O3DE/ROS2 mission sandbox prototype.
- EU-aligned research proposal.

22.9 Build later

Delay until proof loops are stable:

- Full DAO governance.
- High-stakes slashing.
- Large decentralized compute marketplace.
- Full enterprise identity.

- Regulated credentials.
- Large cross-world inventory.
- Full autonomous-world ecosystem.
- Public-sector production deployments.

---

23. Metrics

23.1 Product metrics

- Number of completed tasks.
- Number of validated results.
- Verification confidence distribution.
- Dispute rate.
- Reversal rate.
- Time to first task completion.
- Walletless-to-SpeXID conversion.
- Retention by task type.
- Number of proof environments.
- Number of creators publishing tasks.

23.2 Reputation metrics

- Verified completions per participant.
- Domain-specific success rates.
- Failure-mode distribution.
- Validator accuracy.
- Validator agreement rate.
- Cost-performance profile.
- Reputation portability usage.
- Number of profiles queried by businesses or agents.

23.3 Technical metrics

- ROD anchor success rate.
- Average anchor size.
- Relay uptime.
- Evidence archive availability.
- Recompute success rate.
- Fraud detection rate.
- API latency.
- Task schema version adoption.

23.4 Business metrics

- Paid task volume.

- Verified outcome purchases.
- Enterprise pilot count.
- Creator revenue.
- Validator payouts.
- Dataset/report revenue.
- ROD utility transactions tied to task activity.
- Institutional partnerships.

23.5 Ecosystem metrics

- Active developers.
- SDK downloads.
- Templates published.
- SpeXcode-generated tasks.
- Independent relays.
- Independent validators.
- Open-source contributors.
- Research citations or papers.

---

24. Strategic conclusion

OpenSpeX should not be launched as a vague rebrand, a token story, a giant metaverse promise, or a magical AI verification system. It should be launched as the opening of the SpaceXpanse stack into a clearer infrastructure category: verified digital performance.

The world does not need another platform full of claims. It needs better ways to answer a simple question: who or what can actually deliver?

OpenSpeX answers by building the path:

```
Prove ability.
Build trust.
Earn opportunity.
```

SpaceXpanse provides the foundation: ROD, names, ownership, identity anchors, simulation heritage, developer tools, communication rails, and creator infrastructure. OpenSpeX activates that foundation by coordinating tasks, evidence, validation, reputation, settlement, and opportunity.

Void Runner Arena can prove the fast browser-native loop. Orbit Challenge can prove the serious simulation loop. SpeXcode can generate new tasks and environments. ANGI can make environments agent-native. SpeXID can make identity portable. ROD can anchor settlement and proof. Nostr and Metalog can distribute events. D.A.R.M.A. can make the system understandable and usable.

The long-term end state is not one giant world. It is many worlds, tasks, agents, services, validators, creators, businesses, and institutions sharing common infrastructure for identity, proof, reputation, ownership, discovery, communication, and settlement.

The future should not belong to those who only claim value. It should belong to those who can prove they created it.

If it can be proven, it can be rewarded.

---

25. Appendices

Appendix A: Source map

Internal sources

[I-MANIFESTO] OpenSpeX Manifesto, uploaded project source. Key ideas: proof -> reputation -> opportunity; claims are cheap; identity, ownership, work, skill, contribution, performance should be provable; trust from proven results; if it can be proven, it can be rewarded.

[I-OVERVIEW] OpenSpeX Overview files, uploaded project sources. Key ideas: opportunity network powered by proven performance; open flight recorder for digital work; platform reset problem; audiences; proof environments as examples, not product itself.

[I-POSITIONING] Describing OpenSpeX Effectively files, uploaded project sources. Key ideas: opportunity should lead; proof/reputation are machinery; credible public phrasing; avoid overclaiming universal verification.

[I-MOTTO] Describing OpenSpeX Effectively, uploaded project source. Key ideas: Prove. Build. Earn.; Prove ability. Build trust. Earn opportunity.

[I-FLIGHT] Describing OpenSpeX Effectively and OpenSpeX Overview, uploaded project sources. Key idea: aviation/flight recorder analogy; Void Runner Arena and Orbit Challenge as digital flight tests.

[I-EVOLUTION] SpaceXpanse Evolution, uploaded project source. Key ideas: SpaceXpanse as infrastructure first; OpenSpeX as layer above SpaceXpanse; identity -> task -> evidence -> verification -> reputation -> settlement.

[I-LAUNCH] OpenSpeX Launch Strategy, uploaded project source. Key ideas: brand hierarchy, market category, launch sequencing, avoid overclaims, proof loops, developer stack, microtasks, AI-agent reputation, institutional narrative.

[I-MVP] OpenSpeX MVP Launch Plan, uploaded project source. Key ideas: client claim vs relay validation vs ROD anchor; run claim, validation receipt, reputation delta, compact commitments.

[I-ROD-SPECS] SpaceXpanse ROD blockchain specs, uploaded project source. Key facts: Namecoin-derived consensus, PoW algorithms, merged mining, block time, confirmation parameters, name-value database, games/app interactions.

[I-VOID] Void Runner Arena Naming and OpenSpeX Launch Strategy, uploaded project sources. Key ideas: Void Runner Arena as first browser-native proof environment and human-vs-AI performance arena.

[I-SPEXCODE] SpeXcode Multiverse Expansion and OpenSpeX MVP concept files, uploaded project sources. Key ideas: SpeXcode as creation/deployment layer for tasks, simulations, worlds, assets, AI workflows, autonomous environments.

[I-ANGI] Branch - AI Navigation Layer for Web files, uploaded project sources. Key ideas: Agent Native Graph Interface; OASF, ANGI, GraphRAG, A2A, MCP, OpenSpeX, ROD architecture.

[I-ROD-NOSTR] ROD and Nostr Integration, uploaded project source. Key idea: ROD as authority/discovery/routing; Nostr as transport/delivery.

[I-RODNS] ROD Consensus Analysis and dDNS in SpaceXpanse files, uploaded project sources. Key ideas: .spex and .rod split; ROD stores the record; .spex is public discovery; relays identified by ROD names and pubkeys, not domains.

[I-O3DE-STORM] STORM Mission Simulation Collaboration and SpaceXpanse Outline with O3DE, uploaded project sources. Key ideas: O3DE as high-fidelity 3D/robotics/digital-twin layer; ROS2 bridge; mission telemetry and institutional proof.

External sources

[E-SPACE-ROD] SpaceXpanse official ROD coin page. Key facts: ROD native cryptocurrency and blockchain/core wallet layer; value/data storage; transactions; mining; network connectivity; tokens/NFTs; names/digital IDs; data storage; atomic trading; Play-to-Earn WIP. URL: https://www.spacexpanse.org/rod-coin.html

[E-SPACE-DOCS] SpaceXpanse official documentation and simulator documentation. Key facts: ROD blockchain as decentralized data store; SpaceXpanse components; simulator based on Newtonian mechanics. URLs: https://www.spacexpanse.org/documentation.html and https://www.spacexpanse.org/metaverse-simulator.html

[E-NAMECOIN] Namecoin official site. Key facts: Bitcoin-derived key/value registration and transfer; decentralized DNS; merged mining; censorship-resistant names. URL: https://www.namecoin.org/

[E-XID] Xaya XID repository. Key facts: blockchain names as secure digital identities; login with XAYA; passwordless authentication; owner-controlled metadata and signer addresses. URL: https://github.com/xaya/xid

[E-NOSTR] Nostr protocol site and repository. Key facts: signed events transmitted by relays; open protocol; no trusted central server. URLs: https://nostr.com/ and https://github.com/nostr-protocol/nostr

[E-OIDC] OpenID Connect Core 1.0 specification. Key facts: identity layer on OAuth 2.0; clients verify identity based on authorization-server authentication and receive profile claims. URL: https://openid.net/specs/openid-connect-core-1_0.html

[E-ERC8004] EIP-8004: Trustless Agents. Key facts: identity, reputation, and validation registries for discovering and trusting AI agents in untrusted settings. URL: https://eips.ethereum.org/EIPS/eip-8004

[E-A2A] A2A protocol specification. Key facts: agent cards, tasks, lifecycle, messages, artifacts, task updates, agent-to-agent coordination. URL: https://a2a-protocol.org/latest/specification/

[E-MCP] Model Context Protocol documentation. Key facts: open standard for connecting AI applications to external tools, data, and workflows. URL: https://modelcontextprotocol.io/docs/getting-started/intro

[E-OASF] Open Agentic Schema Framework documentation. Key facts: schema system for agent capabilities, interactions, metadata, and discovery. URL: https://docs.agntcy.org/oasf/open-agentic-schema-framework/

[E-GRAPHRAG] Microsoft GraphRAG documentation. Key facts: knowledge graph extraction, community summaries, local/global/DRIFT/basic search for complex reasoning. URL: https://microsoft.github.io/graphrag/

[E-O3DE] Open 3D Engine GitHub repository. Key facts: open-source, real-time, multi-platform 3D engine for AAA games, cinema-quality worlds, and high-fidelity simulations; Apache-2.0. URL: https://github.com/o3de/o3de

[E-EU-DD] European Commission Digital Decade policy page. Key facts: human-centric sustainable digital transformation, targets for connectivity, digital skills, digital business, public services. URL: https://digital-strategy.ec.europa.eu/en/policies/europes-digital-decade

[E-EU-AI] European Commission AI Factories page. Key facts: AI Factories as ecosystems combining computing power, data, talent, supercomputing centers, universities, SMEs, industry, and other actors. URL: https://digital-strategy.ec.europa.eu/en/policies/ai-factories

[E-OPENAI-SWE] OpenAI article on SWE-bench Verified. Key facts: benchmark no longer used by OpenAI as primary progress metric due to flawed tests and contamination concerns. URL: https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/

[E-OCCUBENCH] OccuBench arXiv paper. Key facts: benchmark for real-world professional tasks, many occupational scenarios, robustness and task completion. URL: https://arxiv.org/abs/2604.10866

[E-JOBBENCH] JobBench arXiv paper. Key facts: benchmark for workflows experts want delegated to AI agents across occupations; current agents remain limited. URL: https://arxiv.org/abs/2604.00344

Appendix B: Current, MVP, planned, experimental, and vision status

Component	Status	Notes
ROD blockchain	Current foundation	Existing technical base, specs, docs, wallet/node foundation.
ROD name-value storage	Current foundation	Core primitive for names, identity anchors, discovery, compact commitments.
SpaceXpanse documentation	Current foundation	Developer docs, ROD specs, tutorials, simulator docs.
Metaverse Simulator	Current foundation	Physics/simulation heritage, education and mission proof potential.
Void Runner Arena	MVP / proof environment	First low-friction proof loop; needs validated commitment layer.
Orbit Challenge	MVP / planned	Serious simulation proof; needs mission rules and telemetry validation.
SpeXID	MVP / planned	Identity/passport layer for humans, agents, validators, creators, providers.
Launchpad dashboard	MVP / planned	Lightweight web dashboard should precede full desktop maturity.
OpenSpeX reputation schema	MVP / planned	Needed for first proof loop.
RODNS / .spex resolver	Planned	Should start as local resolver + JSON API, then expand.
SpeXcode	Planned / strategic	Creation and deployment layer; prototype after first proof loop.
Human microtask market	Planned	Bootstrap demand and validation; avoid generic low-value gig framing.

Component	Status	Notes
AI Runtime / USR service market	Planned / experimental	Controlled PoCs first, no broad claims.
ANGI	Research / planned	Strong strategic direction; start with profile for one environment.
A2A / MCP / OASF integrations	Research / planned	Use as external standards/patterns where appropriate.
O3DE/ROS2 Runtime	Research / planned	Strong institutional path; needs working demo.
Tier 1 high-stakes verification	Experimental	Requires validator consensus, audits, disputes, governance.
Full DAO governance	Vision / later	Delay until usage and stakeholder set are real.
Sovereign institutional deployments	Vision / later	Pilot first; avoid present-tense claims.

Appendix C: Minimum viable OpenSpeX proof loop

1. User or AI agent creates temporary profile or SpeXID.
2. User/agent enters a controlled task environment.
3. Environment generates a task seed and ruleset reference.
4. Performer executes task.
5. Client submits a run claim with evidence hashes.
6. Relay validates the claim under published rules.
7. Relay signs a validation receipt.
8. Reputation delta is calculated.
9. Compact commitment is anchored to ROD or testnet.
10. Public profile displays result, confidence, evidence pointer, and reputation effect.

Appendix D: First 10 recommended deliverables

1. OpenSpeX landing page.
2. Built on SpaceXpanse/ROD explainer.
3. Identity/profile MVP.
4. Task dashboard.
5. Validator UI.
6. Void Runner Arena leaderboard integration.
7. Basic reputation schema.
8. ROD/testnet settlement proof.
9. Public leaderboard/profile view.
10. First Agent Reliability Report.

Appendix E: Glossary

ANGI - Agent Native Graph Interface; machine-readable environment interface for agents. **A2A** - Agent-to-Agent protocol pattern for agent discovery, communication, and collaboration. **D.A.R.M.A.** - Operational assistant for onboarding, support, documentation, validation help, and ecosystem usability. **Metalog** - SpaceXpanse/OpenSpeX communication and event layer direction, related to

Nostr-style relay patterns. **OpenSpeX** - Coordination and reputation layer for verified digital work and opportunity. **Proof environment** - A controlled game, simulation, workflow, task, or service environment where performance can be measured. **ROD** - Native SpaceXpanse blockchain asset and settlement/truth anchor. **RODNS** - ROD-based naming/discovery resolver direction. **SpeXcode** - AI-powered creation and deployment layer for tasks, missions, worlds, assets, workflows, and simulations. **SpeXID** - Identity/passport layer rooted in ROD name ownership and signatures. **USR** - Unified Service Runtime; internal runtime/orchestration layer for routing, validation, settlement, and reputation updates.